Smart Task Manager

FULL-STACK WEB APPLICATION

SYSTEM ARCHITECTURE & PLANNING

Smart Task Manager is a full-stack web application designed to help users organize and manage their daily tasks efficiently. This document outlines the complete project planning and system architecture — covering project objectives, functional and non-functional requirements, user flowcharts, wireframes, technical architecture, database schema, API design, design rationale, and conclusion. The application provides a simple and intuitive interface where users can create, update, delete, and monitor their tasks from a centralized dashboard, with secure user authentication and personalized task lists.

Project Brief & Objectives

Smart Task Manager is a full-stack web application designed to help users organize and manage their daily tasks efficiently. The application provides a simple and intuitive interface where users can create, update, delete, and monitor their tasks from a centralized dashboard. It supports secure user authentication, allowing each user to maintain a personalized task list. Users can categorize tasks, assign priorities, set due dates, and monitor their progress. The primary goal of the application is to improve productivity by providing a structured and organized task management system.

Many individuals struggle to organize their daily work and often forget important deadlines due to poor task management. Existing productivity applications may include unnecessary complexity or paid features that discourage regular use. The Smart Task Manager aims to provide a simple, secure, and user-friendly solution that enables users to effectively organize, prioritize, and monitor their daily tasks without unnecessary complications.

Modern Full-Stack App

Develop a modern full-stack web application using React, Node.js, Express, and MongoDB with a unified JavaScript stack.

Secure Authentication

Implement secure user authentication using JWT tokens and bcrypt password hashing to protect user accounts.

Complete CRUD Operations

Allow users to perform complete create, read, update, and delete operations on tasks through a RESTful API.

Task Categorization & Prioritization

Provide task categorization, priority assignment, and due date management to help users stay organized.

Productivity & Deadlines

Improve productivity through deadline management, progress tracking, and a clean dashboard overview.

Responsive UI

Design a responsive and intuitive user interface with optional dark mode, accessible to all target users.

Practical Experience

Gain practical, hands-on experience in full-stack web development through a real-world project.

The application is intended for students, working professionals, freelancers, small business owners, and individuals managing personal daily activities. Key features include user registration, login, and secure authentication; a centralized dashboard; task creation, editing, and deletion; task categories, due dates, and priorities; search and filter functionality; progress tracking; responsive design; and optional dark mode.

Functional & Non-Functional Requirements

Functional Requirements

User Management

- User Registration & Login
- User Logout
- Password Encryption
- JWT Authentication

Task Management

- Create, Edit & Delete Tasks
- Mark Tasks as Completed
- View, Search, Filter & Sort Tasks
- Assign Categories & Priorities
- Set Due Dates

Dashboard

- Total & Completed Tasks
- Pending Tasks
- Upcoming Deadlines
- Progress Statistics

Non-Functional Requirements

Performance

- Fast page loading
- Efficient API responses
- Optimized database queries

Security

- JWT Authentication
- Password hashing using bcrypt
- Protected API routes
- Input validation
- Secure data transmission

Reliability

- Proper error handling
- Consistent API responses
- Stable database operations

Usability

- Easy navigation
- Responsive interface
- Clean dashboard
- Accessible design

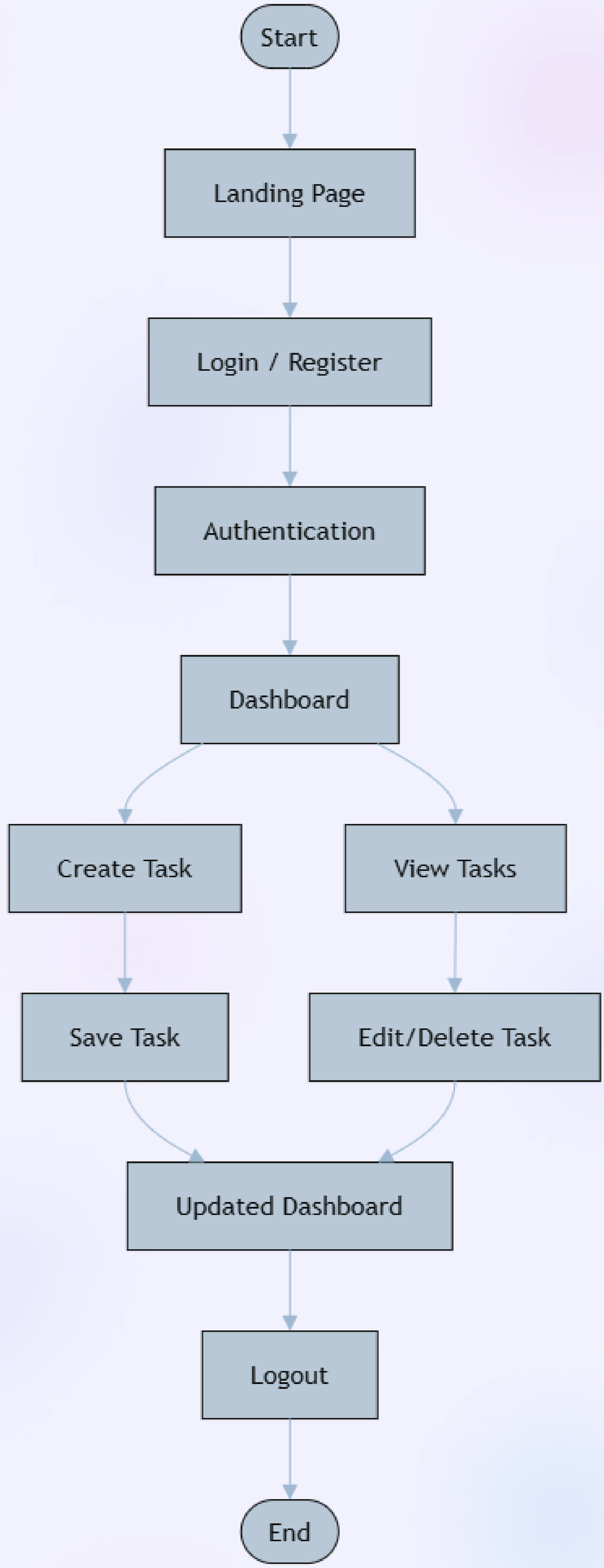
Scalability

- Modular architecture
- RESTful APIs
- Easy feature expansion

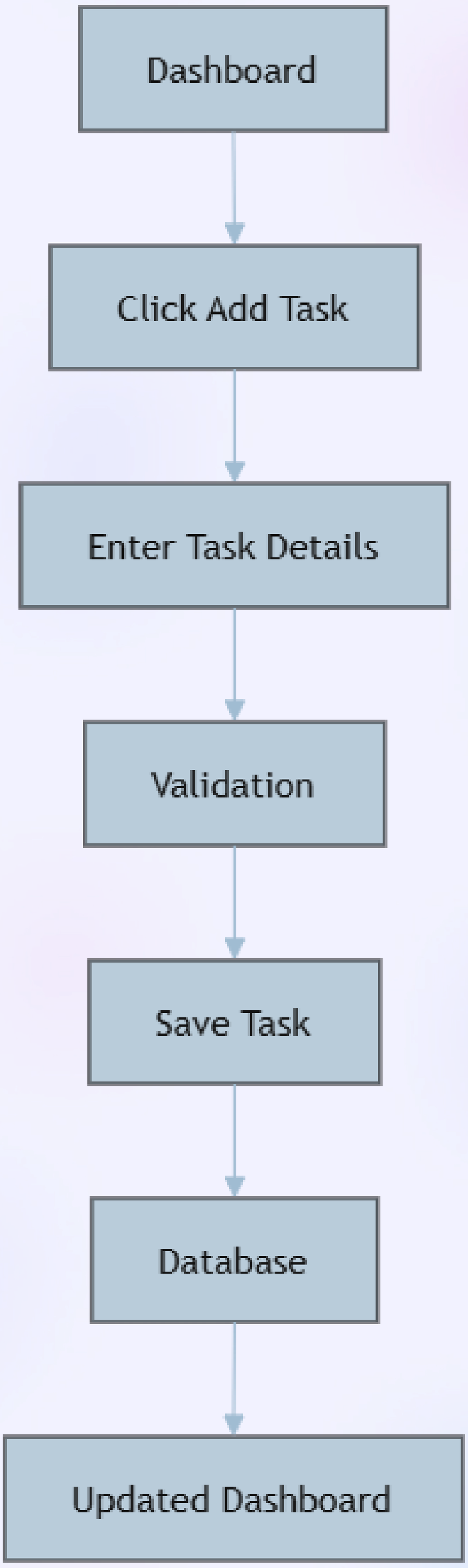
The Smart Task Manager should provide comprehensive functionality across user management, task management, and a centralized dashboard. On the user management side, the application supports user registration, login, logout, password encryption, and JWT authentication to ensure secure access. For task management, users should be able to create a new task, edit an existing task, delete tasks, mark tasks as completed, view all tasks, search tasks, filter tasks, sort tasks, assign categories, set priorities, and set due dates. The dashboard should display total tasks, completed tasks, pending tasks, upcoming deadlines, and progress statistics to give users a clear overview of their workload and productivity.

Non-functional requirements are equally critical to the application's success. Performance requirements include fast page loading, efficient API responses, and optimized database queries. Security is addressed through JWT authentication, password hashing using bcrypt, protected API routes, input validation, and secure data transmission. Reliability is ensured through proper error handling, consistent API responses, and stable database operations. Usability focuses on easy navigation, a responsive interface, a clean dashboard, and accessible design. Finally, scalability is achieved through a modular architecture, RESTful APIs, and an architecture designed for easy feature expansion.

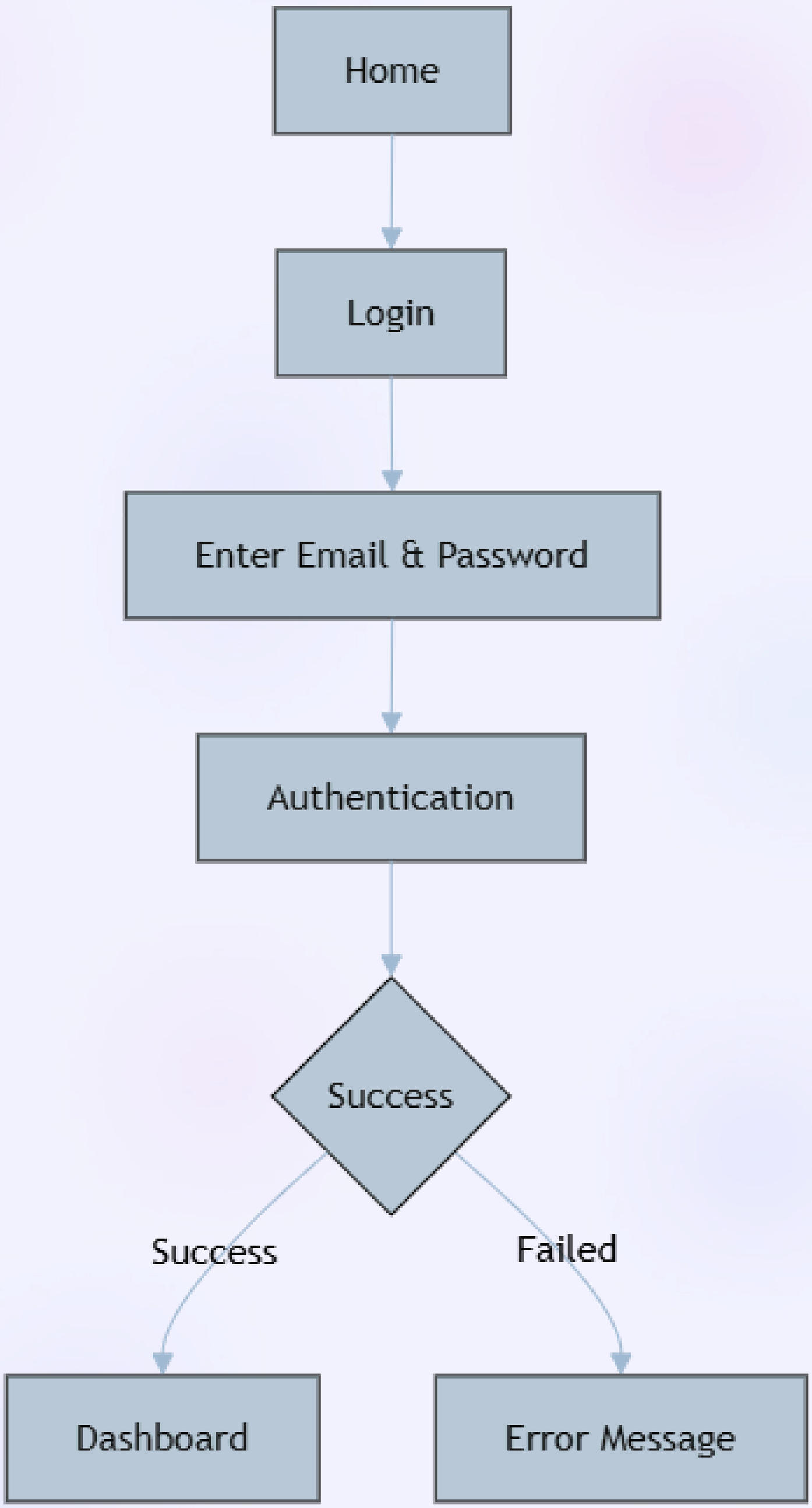
User Flowcharts



Task Creation Flow

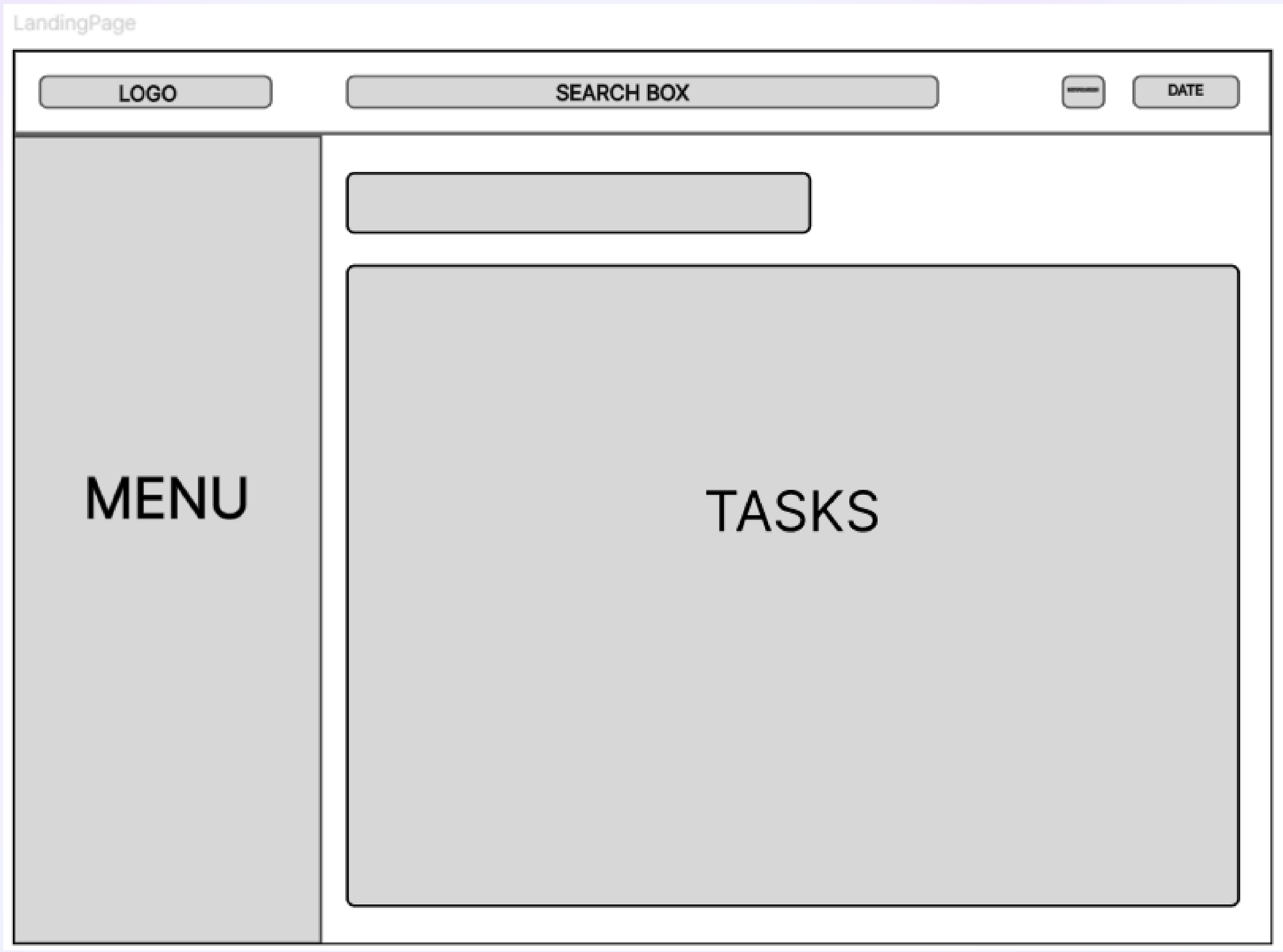


Login Flow

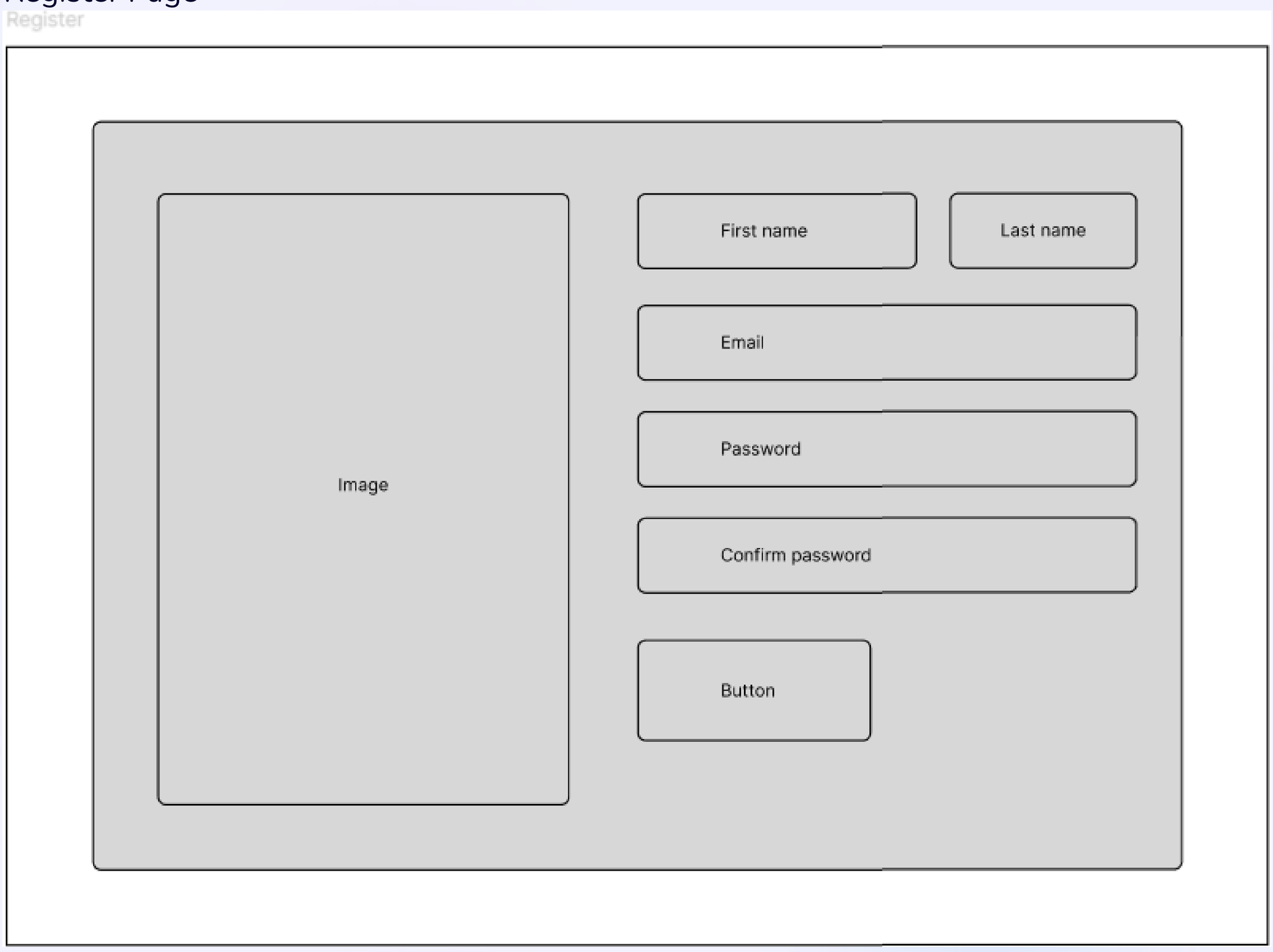

Home
Login
Enter Email & Password
Authentication
Success
Success
Failed
Dashboard
Error Message

Wireframes

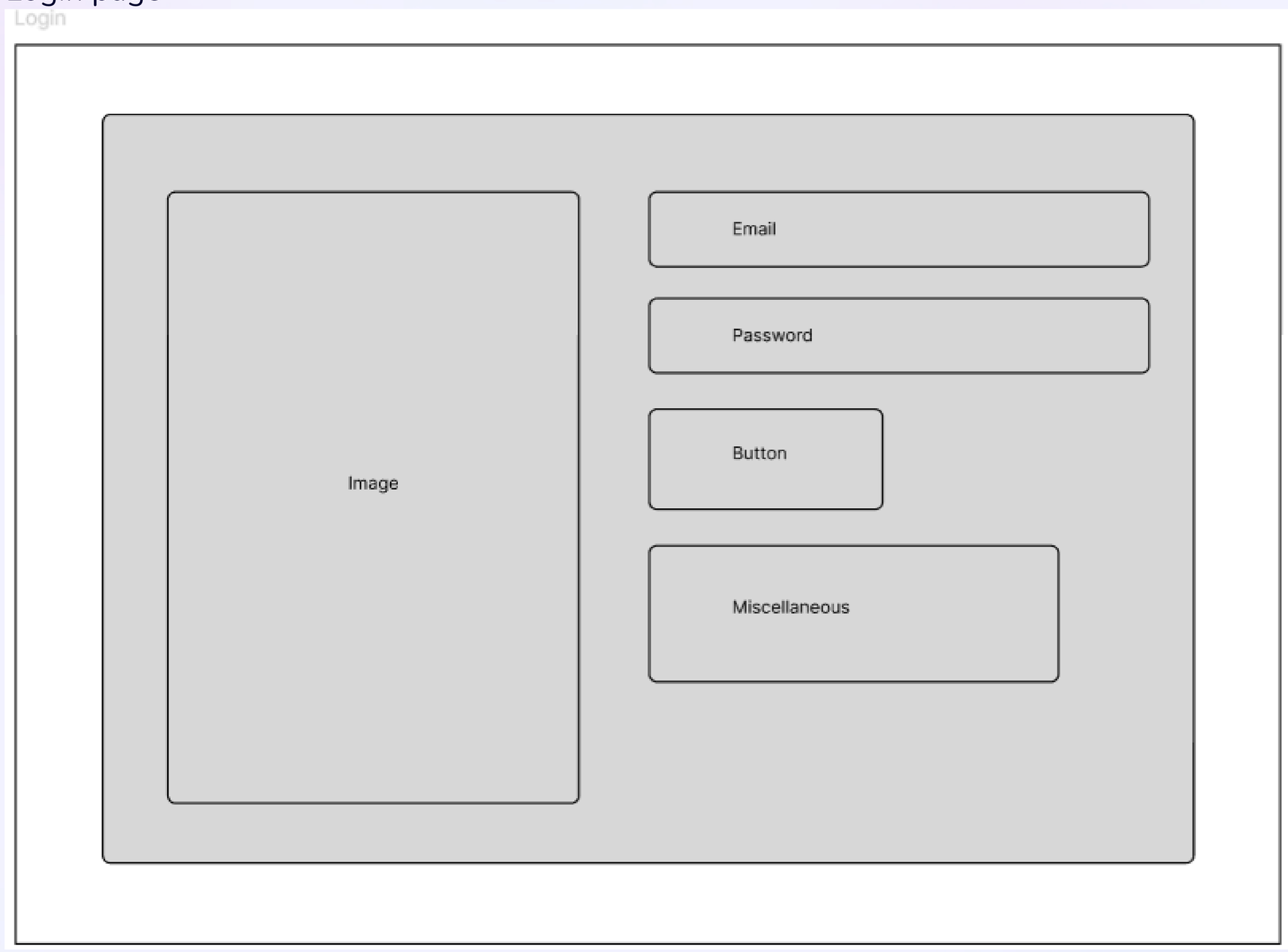
Dashboard



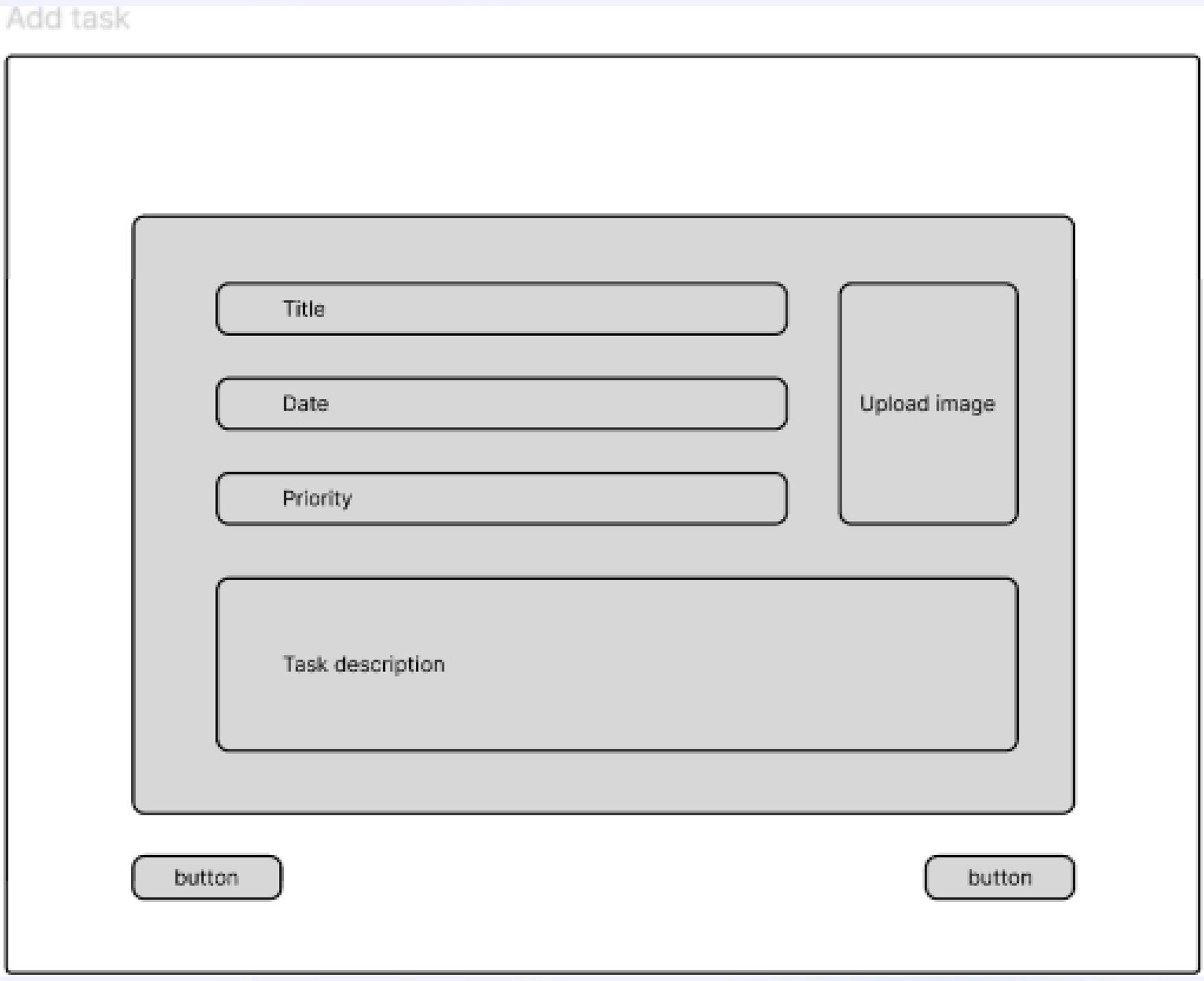
Register Page



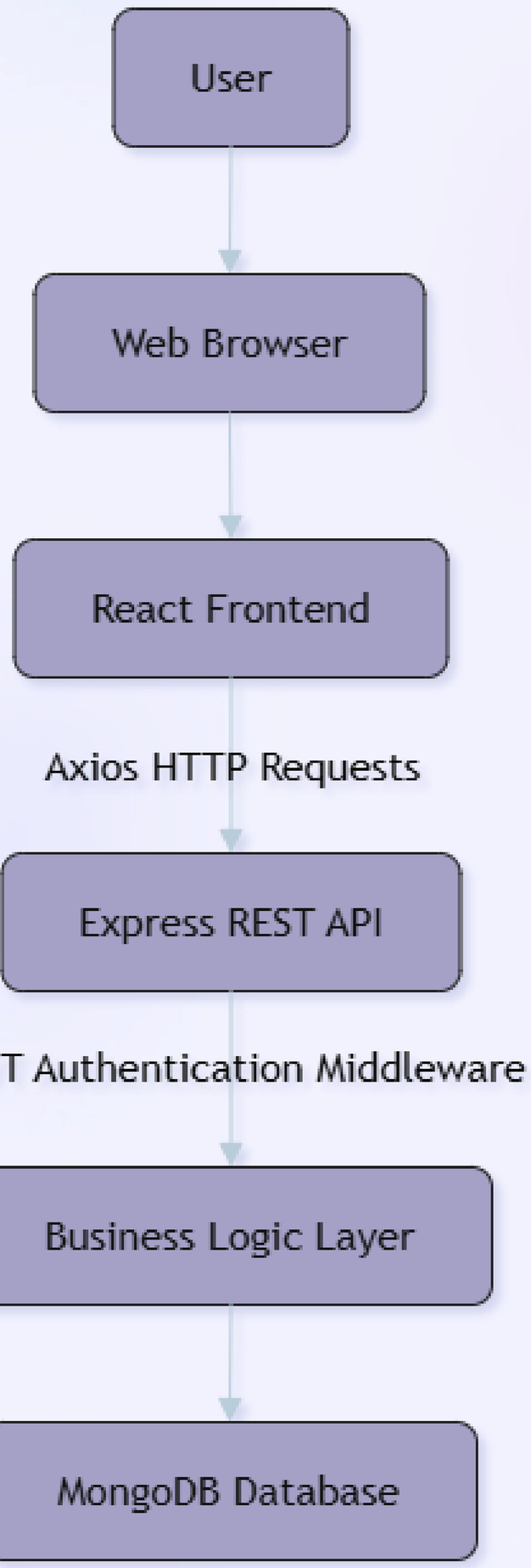
Login page



Add task



Technical Architecture



Component	Technology	Description
Frontend	React, React Router, Axios, CSS / Tailwind CSS	Component-based UI with client-side routing, HTTP requests, and responsive styling
Backend	Node.js, Express.js	High-performance JavaScript runtime with a lightweight RESTful API framework
Authentication	JWT, bcrypt	Stateless token-based authentication with secure password hashing
Database	MongoDB	NoSQL document database with flexible JSON-like storage, well-suited for evolving data structures

Database Schema

The Smart Task Manager uses two primary MongoDB collections: **Users** and **Tasks**. MongoDB stores data in flexible JSON-like documents, which integrates well with Node.js and is suitable for applications where the data structure may evolve over time. The Users collection stores account information with encrypted passwords, while the Tasks collection stores all task-related data linked to the owning user via a foreign key reference (userId). This one-to-many relationship means one user can have multiple tasks, and each task belongs to exactly one user.

Users Collection

Field	Type	Description
_id	ObjectId	Unique ID
name	String	User name
email	String	User email
password	String	Encrypted password
createdAt	Date	Registration date

Tasks Collection

Field	Type	Description
_id	ObjectId	Task ID
title	String	Task title
description	String	Task description
category	String	Task category
priority	String	Low, Medium, High
dueDate	Date	Due date
status	Boolean	Completed / Pending
userId	ObjectId	Owner of task
createdAt	Date	Created date

Database Relationship: One User → Many Tasks. Each task document includes a userId field that references the owning user's _id, enabling efficient querying of a user's personalized task list.

Users Collection

Field	Type	Description
_id	ObjectId	Unique ID
name	String	User name
email	String	User email
password	String	Encrypted password
createdAt	Date	Registration date

_id (ObjectId, unique identifier), **name** (String, user's display name),
email (String, used for login authentication), **password** (String, encrypted using bcrypt for security), and **createdAt** (Date, timestamp of account registration).

Tasks Collection

Field	Type	Description
_id	ObjectId	Task ID
title	String	Task title
description	String	Task description
catagory	String	Task catagory
priority	String	Low, Midium, High
dueDate	Date	Due date
status	Boolean	Completed/Pending
userId	ObjectId	Owner of task
createdAt	Date	Created date

_id (ObjectId, unique task identifier), **title** (String, the task name), **description** (String, detailed task notes),
category (String, for grouping tasks), **priority** (String, with values Low, Medium, or High),
dueDate (Date, deadline for the task), **status** (Boolean, indicating Completed or Pending),
userId (ObjectId, reference to the owning user), and **createdAt** (Date, timestamp of task creation)

API Design

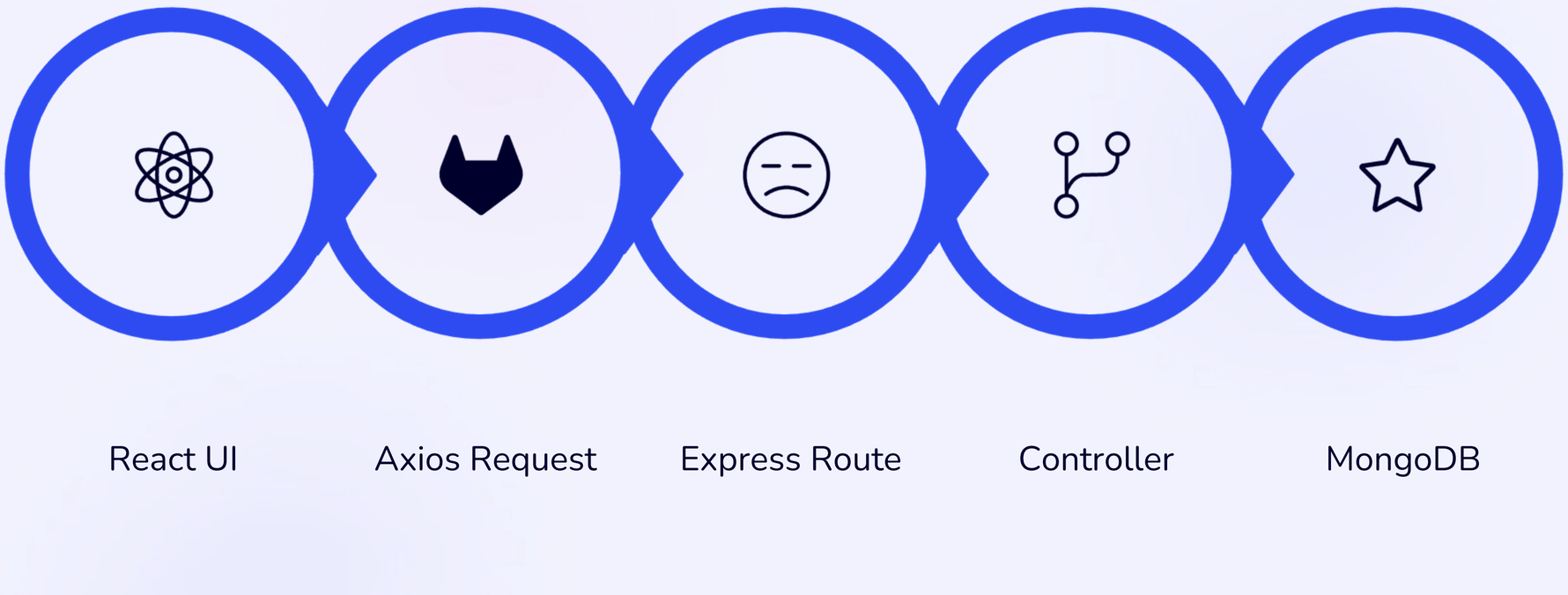
The Smart Task Manager exposes a RESTful API organized into two resource groups: **Authentication APIs** for user registration, login, and profile access, and **Task APIs** for full CRUD operations on tasks. All task-related endpoints are protected by JWT authentication middleware, ensuring that users can only access and modify their own data. The API follows standard HTTP method conventions — POST for creation, GET for retrieval, PUT for updates, and DELETE for removal — returning consistent JSON responses for both success and error cases.

Authentication APIs

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Login user
GET	/api/auth/profile	User profile

Task APIs

Method	Endpoint	Purpose
GET	/api/tasks	Get all tasks
GET	/api/tasks/:id	Get task details
POST	/api/tasks	Create task
PUT	/api/tasks/:id	Update task
DELETE	/api/tasks/:id	Delete task



The example API flow illustrates the complete request-response cycle. When a user performs an action in the React UI — such as creating a new task — Axios sends an HTTP request to the appropriate Express route. The route handler passes the request to a controller, which contains the business logic. The controller interacts with MongoDB to read or write data, then returns a JSON response. Axios receives this response and updates the React UI accordingly, providing immediate feedback to the user. This clean separation of concerns ensures that the frontend remains decoupled from the backend, making the application easier to test, maintain, and extend.

Design Rationale

The Smart Task Manager was selected because task management is a common real-world problem. It demonstrates essential full-stack development concepts — including authentication, CRUD operations, API development, database management, and responsive UI design — while remaining manageable within the project timeline. The following sections explain the rationale behind each technology choice and the overall three-tier architecture.

Why React?

React enables the development of reusable UI components, efficient state management, and fast rendering through its virtual DOM. It is widely adopted in modern web development and simplifies building responsive user interfaces. React's component-based architecture aligns well with the modular design goals of the Smart Task Manager, allowing individual UI elements — such as task cards, forms, and the dashboard — to be developed and tested independently.

Why Node.js & Express?

Node.js provides a high-performance JavaScript runtime, while Express.js offers a lightweight framework for building RESTful APIs. Together, they enable efficient communication between the frontend and backend with a unified JavaScript stack. Using JavaScript across both layers reduces context switching for developers and simplifies the overall technology stack, making the project more approachable for learning full-stack development.

Why MongoDB?

MongoDB is a NoSQL database that stores data in flexible JSON-like documents. It integrates well with Node.js and is suitable for applications where the data structure may evolve over time, making development faster and more adaptable. The document-based model maps naturally to JavaScript objects, reducing the need for complex data transformation between the application and database layers.

Why JWT Authentication?

JWT (JSON Web Token) enables secure, stateless authentication between the client and server. It reduces server-side session management and is widely used in modern web applications. Because JWT tokens are self-contained and can be verified without querying the database on each request, they improve API performance while maintaining security through cryptographic signing.

Overall Architecture

A three-tier architecture — Presentation Layer, Application Layer, and Data Layer — was chosen to ensure modularity, scalability, and maintainability. Separating the frontend, backend, and database allows each layer to be developed, tested, and updated independently, supporting future enhancements. This separation of concerns also makes the codebase easier to understand and extend as new features are added to the application.

Conclusion

The Smart Task Manager project provides a comprehensive foundation for learning full-stack web development. The planning phase establishes clear objectives, system architecture, database design, API specifications, and user interface concepts that will guide the implementation in the following weeks. By following a modular architecture and industry-standard technologies, the application is designed to be secure, scalable, maintainable, and user-friendly.

The project addresses a real-world problem — helping individuals organize their daily work and avoid forgotten deadlines — while demonstrating the essential concepts of modern web application development. The three-tier architecture, RESTful API design, JWT-based authentication, MongoDB schema, and React frontend work together to create a cohesive system that is both functional and educational. Each design decision, from the choice of bcrypt for password hashing to the use of Tailwind CSS for responsive styling, reflects a commitment to industry best practices.

Secure

JWT authentication and bcrypt password hashing protect user data at every layer of the application.

Scalable

Modular three-tier architecture and RESTful APIs support easy feature expansion and independent layer updates.

Maintainable

Clear separation of concerns across frontend, backend, and database layers simplifies testing and future development.

User-Friendly

Responsive design, intuitive navigation, and a clean dashboard ensure an accessible experience for all target users.

The planning documented in this architecture overview — including functional and non-functional requirements, user flowcharts, database schema, API design, and technology rationale — provides a solid blueprint for the implementation phase. As development progresses, the modular structure will allow each component to be built, tested, and refined independently, ensuring that the final application meets the objectives of improving productivity through a structured and organized task management system.